

TP PHP - MVC Boutique en ligne (MVC, POO, PDO, Sessions)

Durée estimée : 3 jours

Type : Solo ou Groupe (2–3)

1. Contexte

Une petite boutique souhaite une application web pour vendre en ligne. L'application doit gérer les produits, les utilisateurs (clients/admin), le panier et les commandes.

Objectif principal : Réaliser une application complète en respectant une architecture professionnelle (MVC) et des normes de sécurité standard.

2. Objectifs pédagogiques

- ☐ Maîtriser PHP (conditions, boucles, formulaires)
- ☐ Utiliser les **sessions** (authentification, panier)
- ☐ Implémenter **PDO** (requêtes préparées, CRUD)
- ☐ Appliquer le pattern **MVC**
- ☐ **POO stricte** (entités = classes typées)
- ☐ Séparation des concepts : Modèle, Manager (ou aussi appelé **Repository**), Contrôleur, Vue

3. Fonctionnalités attendues (Backlog)

3.1 Page d'accueil (Produits)

- ☐ Liste des produits (nom, description, prix)

- ☐ Bouton *Ajouter au panier*
- ☐ (Optionnel) Filtre par nom

3.2 Authentification

- ☐ Formulaire de login (email + mot de passe)
- ☐ Session utilisateur active si succès
- ☐ Message d'erreur si échec
- ☐ Déconnexion
- ☐ **Sécurité** : `password_hash()` et `password_verify()`

3.3 Panier

- ☐ Affichage des produits, quantités, total
- ☐ Modifier les quantités
- ☐ Supprimer un produit
- ☐ Vider le panier
- ☐ Bouton **Valider la commande** (visible uniquement si connecté)

3.4 Passage de commande

- ☐ Création de l'entrée en BDD (`orders`)
- ☐ Création des lignes de commande (`order_items` , prix figé)
- ☐ Vider le panier après validation
- ☐ Redirection vers confirmation ou historique

3.5 Historique des commandes

- ☐ Liste des commandes (date, total)
- ☐ Détail d'une commande (produits achetés)

3.6 Administration (Rôle Admin)

- ☐ Ajouter un produit
- ☐ Modifier un produit
- ☐ Supprimer un produit

4. Contraintes Techniques

Contraintes obligatoires pour valider le TP

- **Base de données** : MySQL
- **Accès aux données** : PDO (requêtes préparées obligatoires)
- **Architecture** : MVC
- **Données temporaires** : Sessions PHP
- **Code** : POO (entités objets, pas de tableaux associatifs métier)
- **Typage strict** : `declare(strict_types=1);`

5. Base de données (Schéma)

Tables

Table	Colonnes principales
users	id, email, password_hash, role (USER / ADMIN)
products	id, name, description, price, created_at
orders	id, user_id, created_at, total
order_items	id, order_id, product_id, quantity, unit_price

Notes

- Ajouter les **clés étrangères**
- Utiliser une **transaction SQL** pour la validation de commande

6. Architecture MVC

```

/boutique/ |— index.php |— config/ | |— Database.php |
|— Router.php |— controller/ | |— ProduitController.php
| |— CommandeController.php | |— UtilisateurController.ph
p | |— AdminController.php |— model/ | |— Produit.php |
| |— Commande.php | |— Utilisateur.php | |— DetailCommande
.php |— repository/ | |— ProduitRepository.php | |— Comm
andeRepository.php | |— UtilisateurRepository.php | |— De
tailCommandeRepository.php |— view/ | |— layout/ | | |—
header.php | | |— footer.php | |— accueil.php | |— login
.php | |— panier.php | |— historique.php | |— admin_prod
uits.php |— public/ | |— css/ | |— js/ | |— sql/ | |— sche
ma.sql

```

7. Règles de conception

1. **Entités (Model)** : propriétés, constructeurs, getters/setters — aucun SQL
2. **Repositories** : SQL uniquement, retournent des objets
3. **Contrôleurs** : orchestration requêtes → données → vues
4. **Vues** : affichage HTML uniquement

8. Planning suggéré

Option Solo (3 jours)

- Jour 1 : BDD, MVC, entités, produits, page accueil
- Jour 2 : Authentification, sessions, panier
- Jour 3 : Commandes, historique, admin CRUD

Option Groupe

- Dev A : Authentification & utilisateurs
- Dev B : Produits & admin CRUD
- Dev C : Panier & tunnel de commande

9. Livrables

- ☐ Code source fonctionnel
- ☐ `sql/schema.sql`
- ☐ `README.md` (installation + compte admin)
- ☐ ON OUBLIE PAS GIT !

Fiche d'aide : Gestion de projet



